

IS311 Web Development

Week 10 - Lecture 11

Dr. Abbas Malik
amaalik@psu.edu.sa



What We're Building: Student Records Manager

A fully functional CRUD web application

A full stack web app to manage student records with:

+ Add Students

Form to insert new student records into MySQL

👁 View All

Bootstrap table listing all students from DB

✎ Edit Records

Update student info with pre-filled form

🗑 Delete Records

Remove students with confirmation prompt

Technology Stack Used:

Front End

HTML5, CSS3, Bootstrap 5, Vanilla JS (Fetch API)

Back End

Node.js, Express.js, RESTful API routes

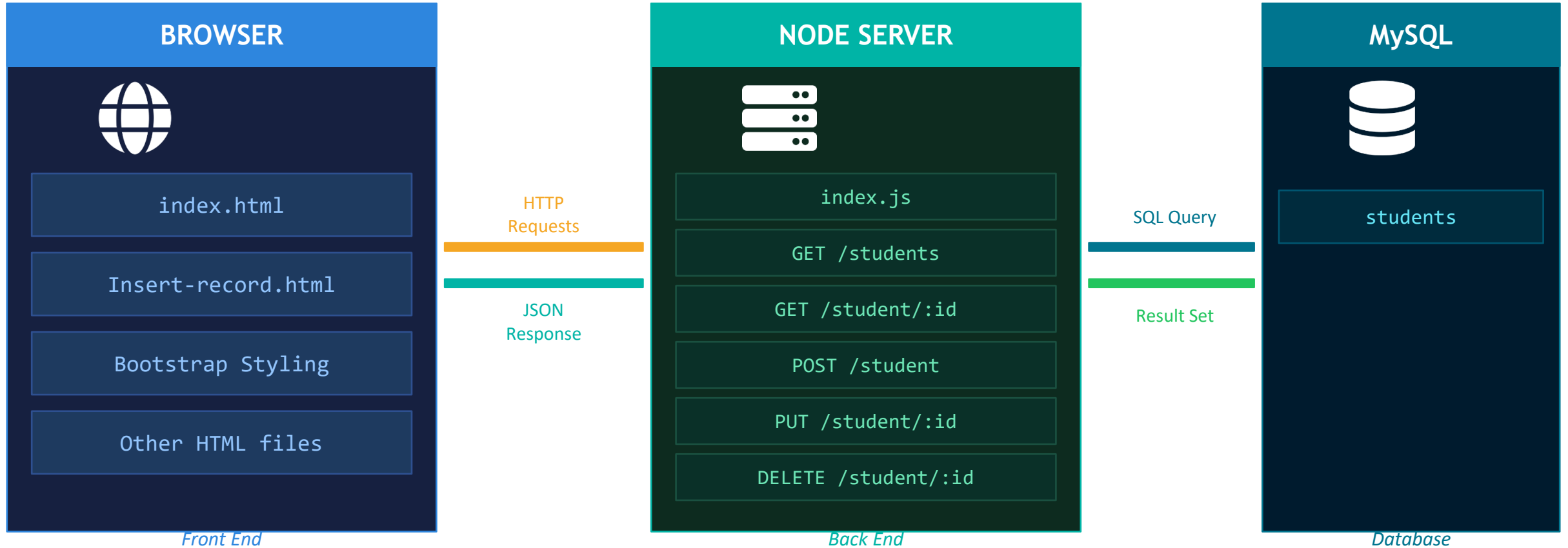
Database

MySQL — students table, CRUD operations



Application Architecture

How the layers connect and communicate





Project Folder Structure

How to organize your files for a clean full stack app

FOLDER TREE

```
dbwebservice/  
├── index.js           ← Entry point (Express server)  
├── package.json      ← Project config &  
dependencies  
└── publichtml/  
    ├── index.html    ← Static front end files  
    │                 ← Main HTML page  
    ├── insert-record.html  
    ├── delete-record.html  
    ├── view-record.html  
    └── bootstrap  
        ├── bootstrap.min.css ← Bootstrap Styles File  
        └── bootstrap.min.js  ← Bootstrap JS File
```

index.js

Initializes Express, connects middleware, mounts the /data/students routes, and starts listening on a port. Defines all 4 CRUD endpoints: GET, POST, PUT, DELETE.

publichtml/index.html

The Bootstrap-based UI. Includes a table to display students and a modal form to add/edit.

publichtml/insert-record.html

Define form to add new record in the backend database server.

Publichtml/view-record.html

Defines how user can access a specific record for an ID and can update its information.

publichtml/delete-record.html

Define how a user can access a record and can delete it.



Step 1: MySQL Database Setup

Creating the database, table, and connection pool

SQL

```
-- Create students table
CREATE TABLE students (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  gender       VARCHAR(10),
  semester    INT NOT NULL,
  major       ENUM('CS', 'IS', 'SE') NOT NULL,
  CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8)
);

-- Optional: Add comments to explain the constraints
ALTER TABLE students
MODIFY COLUMN semester INT NOT NULL COMMENT 'Must be between 1 and 8 inclusive';

ALTER TABLE students
MODIFY COLUMN major ENUM('CS', 'IS', 'SE') NOT NULL COMMENT 'Only CS, IS, or SE allowed';

-- Insert 30 records with Middle Eastern names
INSERT INTO students (name, gender, semester, major) VALUES
-- First 10 students
('Omar Al-Farsi', 'Male', 3, 'CS'),
```

Key Points

- Import SQL file in your DB in SQL Admin
- AUTO_INCREMENT handles the primary key



Step 2: Express Server (index.js)

Wiring up middleware, static files, and routes

index.js

```
// server.js
const express = require('express');
const path    = require('path');
const app     = express();
const PORT    = 3000;

// — Middleware —
app.use(express.json());           // Parse JSON bodies
app.use(express.urlencoded({ extended: true }));

// Serve front end from /public folder
app.use(express.static(__dirname, '/publichtml'));

// — Start Server —
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

`express.json()`

Parses incoming JSON request bodies. Required so POST and PUT routes can read req.body.

`express.urlencoded()`

Parses URL-encoded form data. The `extended:true` option supports rich objects.

`express.static()`

Serves all files in /public directly to the browser — HTML, CSS, and JS.

```
npm install express mysql2
npm install -D nodemon
```



Step 3: API Routes — GET & POST

Reading all students and inserting a new one

index.js
(continued)

```
const db = require('../config/db');

// — GET /data/students — Fetch all students
app.get('/data/students', async (req, res) => {
  try {
    const [rows] = await db.query('SELECT * FROM students');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — POST /data/student — Add a new student
app.post('/data/student', async (req, res) => {
  const { name, email, age, course, gpa } = req.body;
  const sql = 'INSERT INTO students (name,email,age,course,gpa) VALUES (?,?,?,?,?)';
  try {
    const [result] = await db.query(sql, [name, email, age, course, gpa]);
    res.status(201).json({ id: result.insertId, message: 'Student added!' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

GET /data/students

Response: 200 OK → Array of student objects

Runs `SELECT * FROM students`. Returns all rows as JSON array.

POST /data/students

Response: 201 Created → { id, message }

Reads name/email/age/course/gpa from req.body. Uses parameterized query (?) to prevent SQL injection.

⚠ Security Tip

Always use ? placeholders in queries — never concatenate user input directly into SQL strings. This prevents SQL Injection attacks.



Step 3 (cont.): API Routes — PUT & DELETE

Updating and removing student records

index.js
(continued)

```
// — PUT /data/student/:id — Update a student
app.put('/data/student/:id', async (req, res) => {
  const { id } = req.params;
  const { name, email, age, course, gpa } = req.body;
  const sql = `UPDATE students
               SET name=?, email=?, age=?, course=?, gpa=?
               WHERE id=?`;

  try {
    await db.query(sql, [name, email, age, course, gpa, id]);
    res.json({ message: 'Student updated successfully!' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — DELETE /data/student/:id — Remove a student
app.delete('/data/student/:id', async (req, res) => {
  const { id } = req.params;
  try {
    await db.query('DELETE FROM students WHERE id = ?', [id]);
    res.json({ message: 'Student deleted successfully!' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Complete REST API Summary

GET	/data/students	Get all students
POST	/data/student	Add new student
GET	/data/student/:id	Get one student
PUT	/data/student/:id	Update student
DELETE	/data/student/:id	Delete student



req.params vs req.body

req.params.id → from URL /:id

req.body → from JSON request payload



Step 4: Front End — HTML & Bootstrap UI

Building the student management interface

publichtml/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Manager</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.mi
n.css" rel="stylesheet">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container mt-4">
  <h2 class="mb-3">Student Records Manager</h2>

  <!-- Add Student Button -->
  <button class="btn btn-primary mb-3" data-bs-toggle="modal"
    data-bs-target="#studentModal">+ Add Student</button>

  <!-- Students Table -->
  <table class="table table-striped table-hover" id="studentTable">
    <thead class="table-dark">
      <tr>
        <th>ID</th><th>Name</th><th>Email</th>
        <th>Age</th><th>Course</th><th>GPA</th><th>Actions</th>
      </tr>
    </thead>
    <tbody id="tableBody">
```

table-striped / table-hover

Bootstrap table classes for zebra stripes and hover highlight — no custom CSS needed.

table-dark (thead)

Dark header row for visual contrast. Keeps the table scannable for students.

btn btn-primary

Bootstrap button to trigger the Bootstrap Modal for adding/editing students.

Bootstrap Modal

Floating overlay form. Triggered with data-bs-toggle='modal'. Reused for both Add and Edit actions.

💡 Tip: id='tableBody' lets JavaScript target and fill the table rows



Step 5: Client-Side JavaScript (Client Side JS)

Using Fetch API to connect the UI to the Express API

Client JS

```
// public/js/app.js
const API = '/api/students';

// — Load all students on page load
async function loadStudents() {
  const res = await fetch(API);
  const data = await res.json();
  const tbody = document.getElementById('tableBody');
  tbody.innerHTML = '';
  data.forEach(s => {
    tbody.innerHTML += `
      <tr>
        <td>${s.id}</td><td>${s.name}</td>
        <td>${s.email}</td><td>${s.age}</td>
        <td>${s.course}</td><td>${s.gpa}</td>
        <td>
          <button class="btn btn-sm btn-warning"
            onclick="editStudent(${s.id})">Edit</button>
          <button class="btn btn-sm btn-danger"
            onclick="deleteStudent(${s.id})">Delete</button>
        </td>
      </tr>`;
  });
}
```

```
// — Add or Update student (form submit)
async function saveStudent(e) {
```

1

`fetch(url)`

Makes an HTTP request. Default method is GET.

2

`await res.json()`

Parses the JSON response body into a JS object or array.

3

Template Literals

Backtick strings build HTML rows dynamically from API data.

4

method: PUT or POST

Decides based on whether studentId field is filled (edit) or empty (add).

5

Content-Type: JSON

Tell Express the body is JSON so `express.json()` parses it correctly.



Step 5 (cont.): Bootstrap Modal Form

A single modal reused for both Add and Edit operations

index.html –
Modal

```
<!-- Modal inside index.html -->
<div class="modal fade" id="studentModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="modalTitle">Add Student</h5>
        <button type="button" class="btn-close"
          data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <form id="studentForm" onsubmit="saveStudent(event)">
          <input type="hidden" id="studentId">
          <div class="mb-3">
            <label>Name</label>
            <input type="text" class="form-control" id="name" required>
          </div>
          <div class="mb-3">
            <label>Email</label>
            <input type="email" class="form-control" id="email"
required>
          </div>
          <div class="row">
            <div class="col mb-3">
              <label>Age</label>
              <input type="number" class="form-control" id="age"
required>
            </div>
            <div class="col mb-3">
              <label>GPA</label>
              <input type="text" class="form-control" id="gpa" required>
            </div>
          </div>
          <div class="text-center">
            <button type="button" class="btn btn-primary">Save</button>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>
```

Client.js – editStudent()

```
// Pre-fill modal for editing
async function editStudent(id) {
  const res = await fetch(`${API}/${id}`);
  const s = await res.json();

  document.getElementById('studentId').value =
s.id;
  document.getElementById('name').value =
s.name;
  document.getElementById('email').value =
s.email;
  document.getElementById('age').value =
s.age;
  document.getElementById('course').value =
s.course;
  document.getElementById('gpa').value =
s.gpa;

  document.getElementById('modalTitle').value =
s.course + " - " + s.name;
}
```

How the single modal handles both Add & Edit:

The hidden input #studentId is empty for Add (triggers POST) and filled with the row ID for Edit (triggers PUT).



Step 6: Running & Testing the Application

Start the server and verify every feature works

1

Install dependencies

```
npm install express mysql2  
Npm install -D nodemon
```

2

Add start script

```
"dev": "nodemon index.js" (in package.json)
```

3

Start MySQL server

Open MySQL in XAMPP

4

Run the application

```
npm run dev
```

5

Open in browser

```
http://localhost:3000
```

✓ **Test Checklist:**

Load page (GET) ✓ Add student (POST) ✓ Edit student (PUT) ✓ Delete student (DELETE) ✓ Check MySQL for changes



Common Errors & How to Fix Them

Debugging tips every full stack student needs

⚠ Cannot GET /api/students

✓ Fix: Route not mounted. Check `app.use('/api/students', studentRoutes)` in `server.js`

⚠ `req.body` is undefined

✓ Fix: Missing `express.json()` middleware. Add it before your routes.

⚠ ER_ACCESS_DENIED_ERROR (MySQL)

✓ Fix: Wrong username/password in `db.js`. Verify credentials and restart.

⚠ ECONNREFUSED (MySQL not running)

✓ Fix: MySQL server is stopped. Start it via Workbench, XAMPP, or system service.

⚠ CORS error in browser console

✓ Fix: Add `npm install cors`, then `app.use(require('cors')())` in `server.js`

⚠ Table shows old data after edit

✓ Fix: Call `loadStudents()` after every fetch POST/PUT/DELETE to refresh the table.

⚠ Duplicate entry for email

✓ Fix: Unique constraint on email. Add proper error handling and user feedback.



Best Practices for Full Stack Development

Write code that's clean, secure, and maintainable



Database Best Practices

- Always use parameterized queries (?) — never concatenate user input into SQL
- Use connection pools (not single connections) for performance
- Name tables/columns consistently (snake_case)
- Add created_at timestamps to track records



API Best Practices

- Use proper HTTP status codes: 200 OK, 201 Created, 404 Not Found, 500 Error
- Always wrap async routes in try/catch and return error JSON
- Validate input before inserting into the database
- Structure routes in separate files — don't put everything in server.js



Front End Best Practices

- Separate concerns: HTML structure, CSS styling, JS behavior
- Use async/await with Fetch — avoid callback hell
- Always call loadStudents() after every data change
- Give users feedback: alerts, spinners, or success messages



Project Best Practices

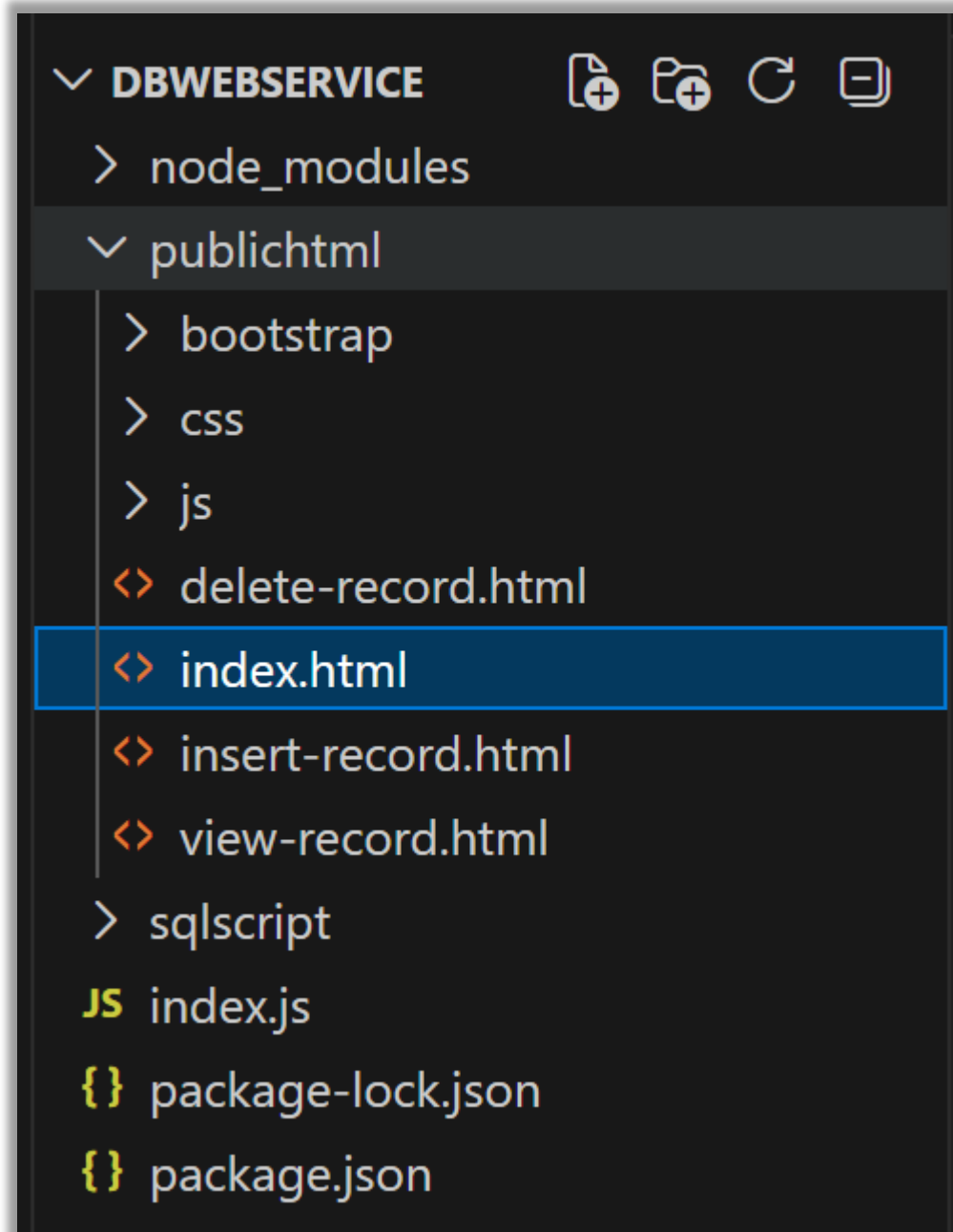
- Store sensitive config in .env files (use dotenv package)
- Add a .gitignore for node_modules and .env
- Test each route with Postman before connecting the front end
- Use nodemon during development to auto-restart on changes

Let us Build it in Detail

- index.html will be served to browser for this
 1. <http://localhost:PORT/>
- We have 'dbwebservice' folder where we have our index.js file that we have create so far.
- To show an HTML page for root route.
- Create a folder 'publichtml' and create HTML files.

Create HTML Files

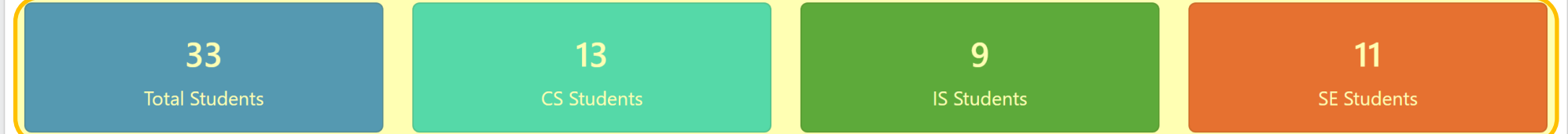
- index.html
- Insert-record.html
- View-record.html
- Delete-record.html



All RecordsView/Edit RecordDelete RecordInsert Record

All Students Records

Search by name, major, or semester...



ID	Name	Gender	Semester	Major	Actions
1	Omar Al-Farsi	Male	Semester 3	CS	EditDelete
2	Fatima Hassan	Female	Semester 5	IS	EditDelete
3	Ahmed Al-Mansouri	Male	Semester 1	SE	EditDelete
4	Layla Abdullah	Female	Semester 7	CS	EditDelete
5	Youssef Khalil	Male	Semester 2	IS	EditDelete
6	Nour El-Din	Female	Semester 4	SE	EditDelete
7	Hassan Al-Rashid	Male	Semester 6	CS	EditDelete
8	Amina Al-Zahrani	Female	Semester 8	IS	EditDelete
9	Kareem Abbas	Male	Semester 3	SE	EditDelete
10	Mariam Al-Saud	Female	Semester 5	CS	EditDelete

index

```
<body class="bg-light">
  <div class="container py-4">
    <!-- Navigation Buttons -->
    <div class="row mb-4">
      <div class="col-12">
        <div class="btn-group w-100" role="group">
          <a href="index.html" class="btn btn-primary active">
            <i class="fas fa-list"></i> All Records
          </a>
          <a href="view-record.html" class="btn btn-outline-primary">
            <i class="fas fa-eye"></i> View/Edit Record
          </a>
          <a href="delete-record.html" class="btn btn-outline-primary">
            <i class="fas fa-trash-alt"></i> Delete Record
          </a>
          <a href="insert-record.html" class="btn btn-outline-primary">
            <i class="fas fa-plus-circle"></i> Insert Record
          </a>
        </div>
      </div>
    </div>
  </div>
```

Index.html JavaScript Code

```
const API_BASE_URL = "http://localhost:3000/data";
let allStudents = [];
// Helper function to show alerts
function showAlert(message, type) {
  const alertDiv = document.createElement("div");
  alertDiv.className = `alert alert-${type} alert-dismissible fade show
position-fixed top-0 end-0 m-3`;
  alertDiv.style.zIndex = "9999";
  alertDiv.style.minWidth = "300px";
  alertDiv.innerHTML = `${message}
  <button type="button" class="btn-close" data-bs-dismiss="alert"></button>`;
  document.body.appendChild(alertDiv);
  setTimeout(() => alertDiv.remove(), 3000);
}
```

Index.html JavaScript Code

```
// Load all students on page load
document.addEventListener("DOMContentLoaded", () => {
  loadStudents();
  document
    .getElementById("searchInput")
    .addEventListener("input", filterStudents);
});
```

Index.html JavaScript Code

```

async function loadStudents() {
  try {
    const response = await fetch(`${API_BASE_URL}/students`);
    const result = await response.json();
    if (result.length) {
      allStudents = result;
      displayStudents(allStudents);
      updateStatistics(allStudents);
    } else {
      document.getElementById("studentTableBody").innerHTML = `
        <tr><td colspan="6" class="text-center text-danger">No students found</td></tr>`;
    }
  } catch (error) {
    showAlert("Error loading students: " + error.message, "danger");
    document.getElementById("studentTableBody").innerHTML = `
      <tr><td colspan="6" class="text-center text-danger">Failed to load students</td></tr>`;
  }
}

```

```

function displayStudents(students) {
  const tableBody = document.getElementById("studentTableBody");
  tableBody.innerHTML = students
    .map(
      (student) => `
        <tr>
          <td><strong>${student.id}</strong></td>
          <td>${escapeHtml(student.name)}</td>
          <td><span class="badge ${student.gender === "Male" ? "bg-info" : "bg-
warning"}">${student.gender || "N/A"}</span></td>
          <td><span class="badge bg-secondary">Semester ${student.semester}</span></td>
          <td><span class="badge ${student.major === "CS" ? "bg-primary" : student.major === "IS" ?
"bg-success" : "bg-danger"}">${student.major}</span></td>
          <td>
            <a href="view-record.html?id=${student.id}" class="btn btn-sm btn-warning">
              <i class="fas fa-edit"></i> Edit
            </a>
            <a href="delete-record.html?id=${student.id}" class="btn btn-sm btn-danger">
              <i class="fas fa-trash"></i> Delete
            </a>
          </td>
        </tr>`,
    )
    .join("");
}

```

Index.html JavaScript Code

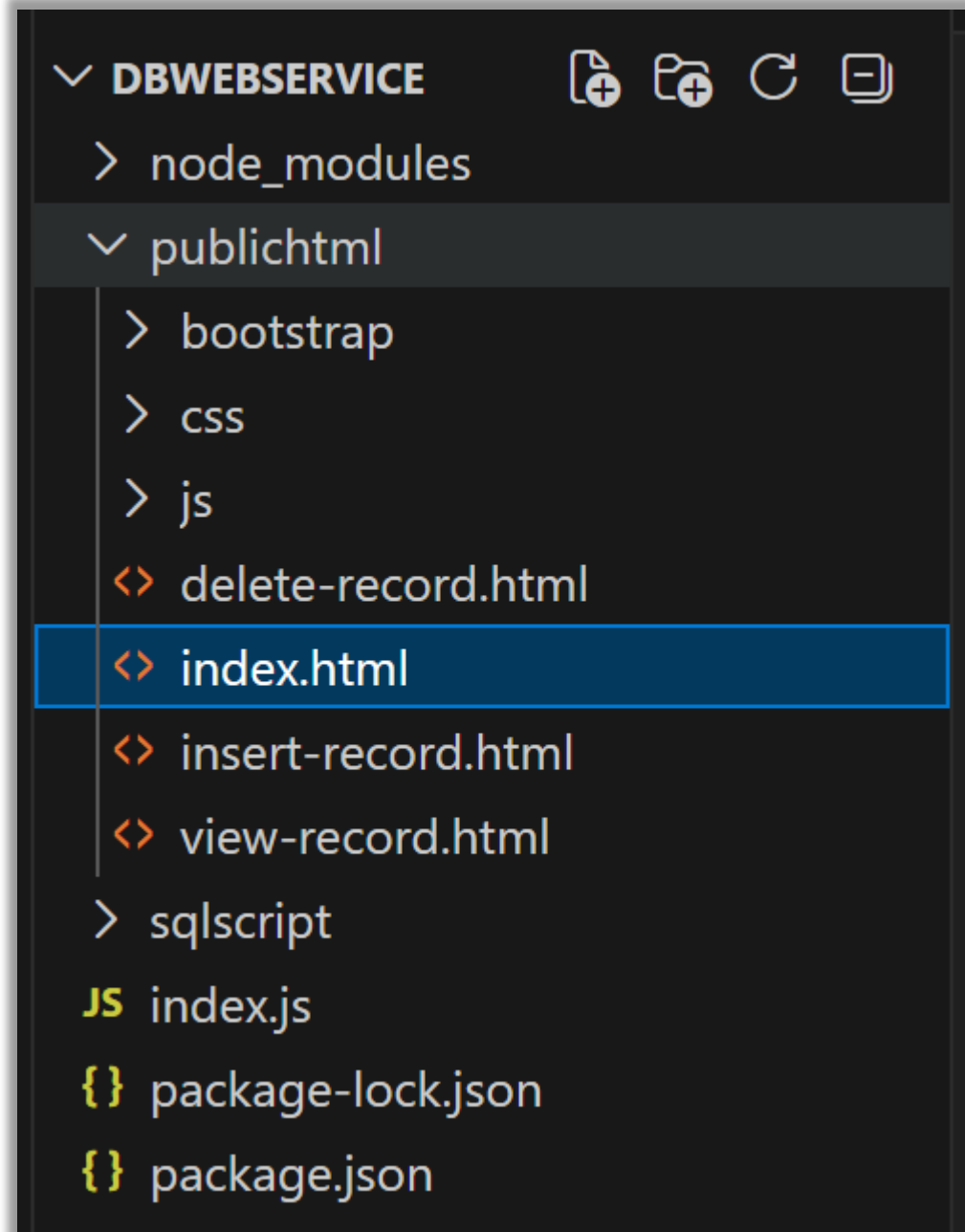
```
function filterStudents() {  
  const searchTerm = document  
    .getElementById("searchInput")  
    .value.toLowerCase();  
  const filtered = allStudents.filter(  
    (student) =>  
      student.name.toLowerCase().includes(searchTerm) ||  
      student.major.toLowerCase().includes(searchTerm) ||  
      student.semester.toString().includes(searchTerm),  
  );  
  displayStudents(filtered);  
}
```


Index.html JavaScript Code

```
function updateStatistics(students) {  
    document.getElementById("totalStudents").textContent =  
students.length;  
    document.getElementById("totalCS").textContent = students.filter(  
        (s) => s.major === "CS",  
    ).length;  
    document.getElementById("totalIS").textContent = students.filter(  
        (s) => s.major === "IS",  
    ).length;  
    document.getElementById("totalSE").textContent = students.filter(  
        (s) => s.major === "SE",  
    ).length;  
}
```

Create HTML Files

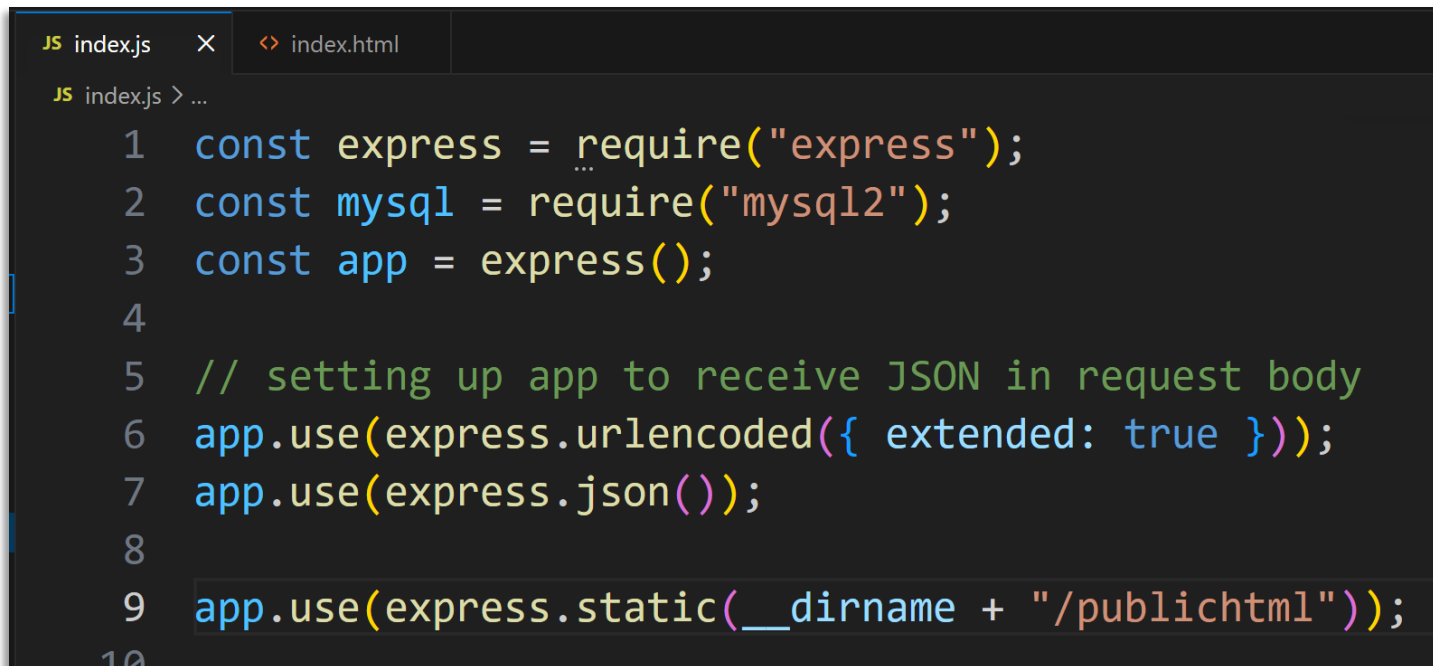
- ✓ index.html
- Insert-record.html
- View-record.html
- Delete-record.html



Serve static contents to Client

- From Express App serve static contents

```
app.use(express.static(__dirname + '/publichtml'));
```



```
JS index.js  X  <> index.html
JS index.js > ...
1  const express = require("express");
2  const mysql = require("mysql2");
3  const app = express();
4
5  // setting up app to receive JSON in request body
6  app.use(express.urlencoded({ extended: true }));
7  app.use(express.json());
8
9  app.use(express.static(__dirname + "/publichtml"));
10
```

All RecordsView/Edit RecordDelete RecordInsert Record

All Students Records

33
Total Students

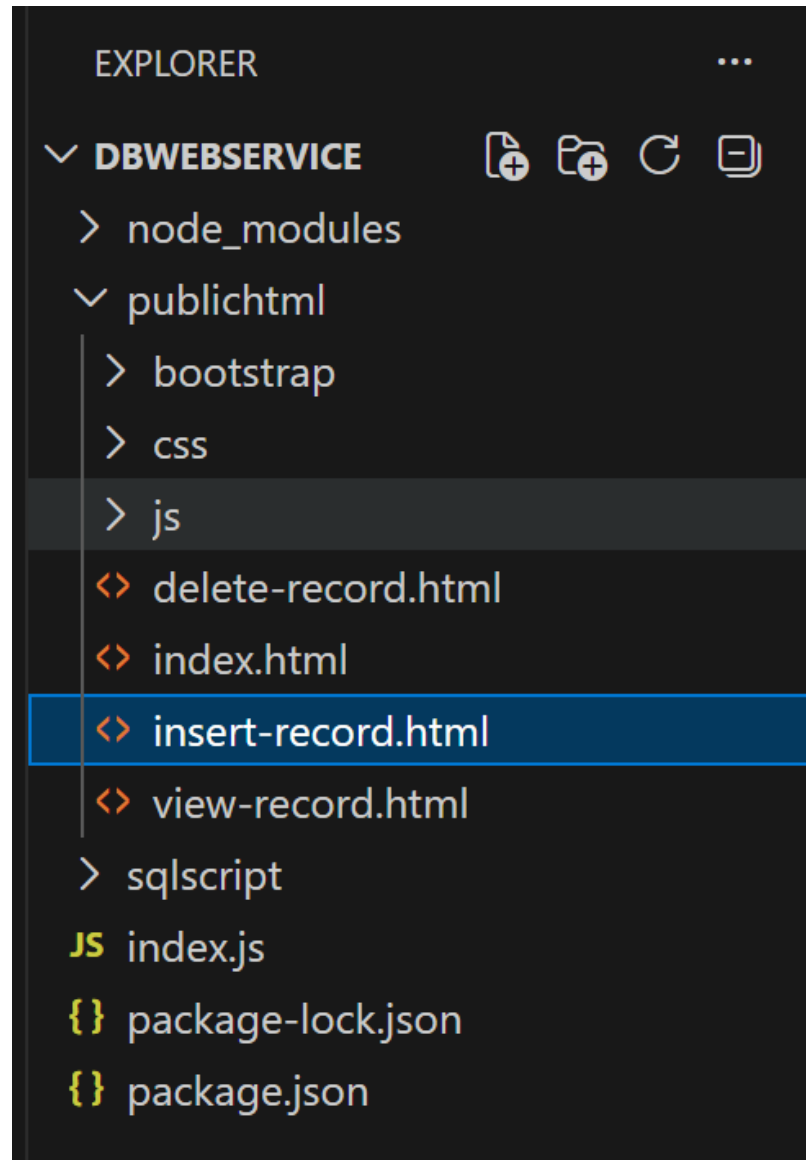
13
CS Students

9
IS Students

11
SE Students

ID	Name	Gender	Semester	Major	Actions
1	Omar Al-Farsi	Male	Semester 3	CS	EditDelete
2	Fatima Hassan	Female	Semester 5	IS	EditDelete
3	Ahmed Al-Mansouri	Male	Semester 1	SE	EditDelete
4	Layla Abdullah	Female	Semester 7	CS	EditDelete
5	Youssef Khalil	Male	Semester 2	IS	EditDelete
6	Nour El-Din	Female	Semester 4	SE	EditDelete
7	Hassan Al-Rashid	Male	Semester 6	CS	EditDelete
8	Amina Al-Zahrani	Female	Semester 8	IS	EditDelete
9	Kareem Abbas	Male	Semester 3	SE	EditDelete
10	Mariam Al-Saud	Female	Semester 5	CS	EditDelete

Insert Record



- All Records
- View/Edit Record
- Delete Record
- Insert Record

Add New Student

Instructions: Fill in the form below to add a new student to the database. All fields marked with * are required.

Full Name *

Example: Omar Al-Farsi

Gender

Semester *

Must be between 1 and 8

Major *

Add Student

Reset Form

Fill Sample Data

```

const API_BASE_URL = "http://localhost:3000/data";

function showAlert(message, type) {
  const alertDiv = document.createElement("div");
  alertDiv.className = `alert alert-${type} alert-dismissible
fade show position-fixed top-0 end-0 m-3`;
  alertDiv.style.zIndex = "9999";
  alertDiv.style.minWidth = "300px";
  alertDiv.innerHTML = `${message}<button type="button"
class="btn-close" data-bs-dismiss="alert"></button>`;
  document.body.appendChild(alertDiv);
  setTimeout(() => alertDiv.remove(), 3000);
}

```

```
function showLoading(show) {
  let spinner = document.getElementById("loadingSpinner");
  if (show && !spinner) {
    spinner = document.createElement("div");
    spinner.id = "loadingSpinner";
    spinner.className =
      "position-fixed top-0 start-0 w-100 h-100 d-flex justify-content-
center align-items-center";
    spinner.style.backgroundColor = "rgba(0,0,0,0.5)";
    spinner.style.zIndex = "9998";
    spinner.innerHTML =
      '<div class="spinner-border text-light" role="status"><span
class="visually-hidden">Loading...</span></div>';
    document.body.appendChild(spinner);
  } else if (!show && spinner) {
    spinner.remove();
  }
}
```



```
document.addEventListener("DOMContentLoaded", () => {
  document
    .getElementById("studentForm")
    .addEventListener("submit", addStudent);

  // Add real-time preview
  ["name", "gender", "semester", "major"].forEach((field) => {
    document.getElementById(field).addEventListener("input", showPreview);
    document
      .getElementById(field)
      .addEventListener("change", showPreview);
  });
});
```

```
async function addStudent(event) {
  event.preventDefault();

  const studentData = {
    name: document.getElementById("name").value.trim(),
    gender: document.getElementById("gender").value,
    semester: parseInt(document.getElementById("semester").value),
    major: document.getElementById("major").value,
  };

  if (!studentData.name) {
    showAlert("Please enter student name", "warning");
    return;
  }
  if (studentData.semester < 1 || studentData.semester > 8) {
    showAlert("Semester must be between 1 and 8", "warning");
    return;
  }

  try {
    showLoading(true);
```

```
function resetForm() {  
    document.getElementById("studentForm").reset();  
    document.getElementById("gender").value = "Male";  
    document.getElementById("major").value = "CS";  
    document.getElementById("previewCard").style.display = "none";  
}
```

```
fillSampleData()
```

```
showPreview()
```

Add POST '/data/students' Route

```
app.post("/data/student", (req, res) => {
  const student = {
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `INSERT INTO students (name, gender, semester, major) VALUES
    ('${student.name}', '${student.gender}', ${student.semester}, '${student.major}')`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    // res.send("<h2>One record is added.</h2>");
    res.status(200).send({ success: true, data: student });
  });
});
```

All Records
View/Edit Record
Delete Record
Insert Record

All Students Records

Search by name, major, or semester...

33
Total Students

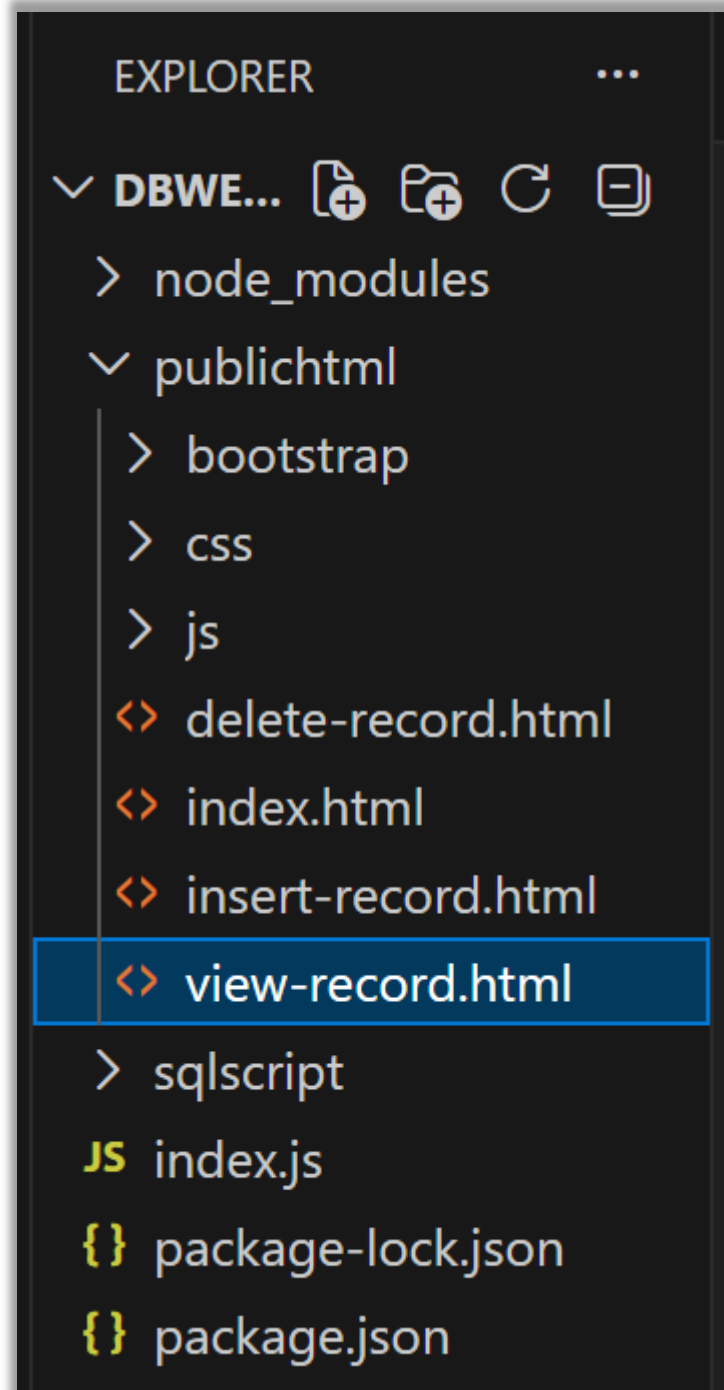
13
CS Students

9
IS Students

11
SE Students

ID	Name	Gender	Semester	Major	Actions
1	Omar Al-Farsi	Male	Semester 3	CS	Edit Delete
2	Fatima Hassan	Female	Semester 5	IS	Edit Delete
3	Ahmed Al-Mansouri	Male	Semester 1	SE	Edit Delete
4	Layla Abdullah	Female	Semester 7	CS	Edit Delete
5	Youssef Khalil	Male	Semester 2	IS	Edit Delete
6	Nour El-Din	Female	Semester 4	SE	Edit Delete
7	Hassan Al-Rashid	Male	Semester 6	CS	Edit Delete
8	Amina Al-Zahrani	Female	Semester 8	IS	Edit Delete
9	Kareem Abbas	Male	Semester 3	SE	Edit Delete
10	Mariam Al-Saud	Female	Semester 5	CS	Edit Delete

View/Update Record



View/Update Record

localhost:3000/view-record.html

★ Bookmarks Fun Courses KSA-Banks KAU Jobs Islam Journals Google Translate Suggested Sites References SDN NanoNetworks NZ Life NLP All Bookmarks

All Records View/Edit Record Delete Record Insert Record

View / Edit Student Record

Enter Student ID:

Enter student ID (e.g., 1, 2, 3...) Load Record

Student Management System | Full Stack Application

View/Update Record

localhost:3000/view-record.html

Bookmarks
Fun
Courses
KSA-Banks
KAU
Jobs
Islam
Journals
Google Translate
Suggested Sites
References
SDN
NanoNetworks
NZ Life
NLP
All Bookmarks

All Records
View/Edit Record
Delete Record
Insert Record

View / Edit Student Record

Enter Student ID:
35
Load Record

Full Name *
Hassan Al-Rashid
Gender
Male

Semester *
4
Major *
Software Engineering (SE)

Must be between 1 and 8

Update Student
Clear

Record Loaded: You can now edit the student information above and click Update to save changes.

Student Management System | Full Stack Application

View/Update Record

```
const API_BASE_URL = "http://localhost:3000/data";
let currentStudentId = null;

function showAlert(message, type) {
  const alertDiv = document.createElement("div");
  alertDiv.className = `alert alert-${type} alert-dismissible fade show position-
fixed top-0 end-0 m-3`;
  alertDiv.style.zIndex = "9999";
  alertDiv.style.minWidth = "300px";
  alertDiv.innerHTML = `${message}<button type="button" class="btn-close" data-bs-
dismiss="alert"></button>`;
  document.body.appendChild(alertDiv);
  setTimeout(() => alertDiv.remove(), 3000);
}
```

View/Update Record

```
function showLoading(show) {
  let spinner = document.getElementById("loadingSpinner");
  if (show && !spinner) {
    spinner = document.createElement("div");
    spinner.id = "loadingSpinner";
    spinner.className =
      "position-fixed top-0 start-0 w-100 h-100 d-flex justify-content-center align-
items-center";
    spinner.style.backgroundColor = "rgba(0,0,0,0.5)";
    spinner.style.zIndex = "9998";
    spinner.innerHTML =
      '<div class="spinner-border text-light" role="status"><span class="visually-
hidden">Loading...</span></div>';
    document.body.appendChild(spinner);
  } else if (!show && spinner) {
    spinner.remove();
  }
}
```

View/Update Record

```
document.addEventListener("DOMContentLoaded", () => {  
    const urlParams = new URLSearchParams(window.location.search);  
    const id = urlParams.get("id");  
    if (id) {  
        document.getElementById("studentId").value = id;  
        loadStudent();  
    }  
    document  
        .getElementById("studentForm")  
        .addEventListener("submit", updateStudent);  
});
```

View/Update Record

```
async function loadStudent() {  
  const id = document.getElementById("studentId").value;  
  if (!id) {  
    showAlert("Please enter a student ID", "warning");  
    return;  
  }  
  
  try {  
    showLoading(true);  
    const response = await fetch(`${API_BASE_URL}/student/${id}`);  
    const result = await response.json();  
    if (result.success) {  
      currentStudentId = result.data.id;  
      displayStudentInForm(result.data);  
      showAlert("Student record loaded successfully", "success");  
    } else {  
      throw new Error(result.message);  
    }  
  }  
}
```

View/Update Record

```
function displayStudentInForm(student) {  
    document.getElementById("studentIdField").value = student.id;  
    document.getElementById("name").value = student.name;  
    document.getElementById("gender").value = student.gender || "Male";  
    document.getElementById("semester").value = student.semester;  
    document.getElementById("major").value = student.major;  
    document.getElementById("studentFormContainer").style.display =  
    "block";  
}
```

Server Edited

```
app.get("/data/student/:id", (req, res) => {  
  let sqlQuery = `SELECT * FROM students WHERE id=${req.params.id}`;  
  dbCon.query(sqlQuery, (error, results, fields) => {  
    if (error) {  
      res.status(500).send({ message: "Internal Server Error.." });  
      throw error;  
    }  
    res.send({ success: true, data: results[0] });  
  });  
});
```

Server Edited

```
app.put("/data/student/:id", (req, res) => {
  const student = {
    id: req.params.id,
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `UPDATE students
  Set
  name = '${student.name}',
  gender = '${student.gender}',
  semester = ${student.semester},
  major = '${student.major}'
  WHERE id = ${student.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) {
      res.status(500).send({ message: "Internal Server Error.." });
      throw error;
    }
    // res.send("<h2>One record is updated.</h2>");
    res.status(200).send({ success: true, data: student });
  });
});
```

Update Record - Client Side

```
async function updateStudent(event) {  
  event.preventDefault();  
  const studentData = {  
    name: document.getElementById("name").value,  
    gender: document.getElementById("gender").value,  
    semester: parseInt(document.getElementById("semester").value),  
    major: document.getElementById("major").value,  
  };  
  
  if (!studentData.name) {  
    showAlert("Name is required", "warning");  
    return;  
  }  
  
  if (studentData.semester < 1 || studentData.semester > 8) {  
    showAlert("Semester must be between 1 and 8", "warning");  
    return;  
  }  
  
  try {  
    // ...  
  }  
}
```


All Records

View/Edit Record

Delete Record

Insert Record

View / Edit Student Record

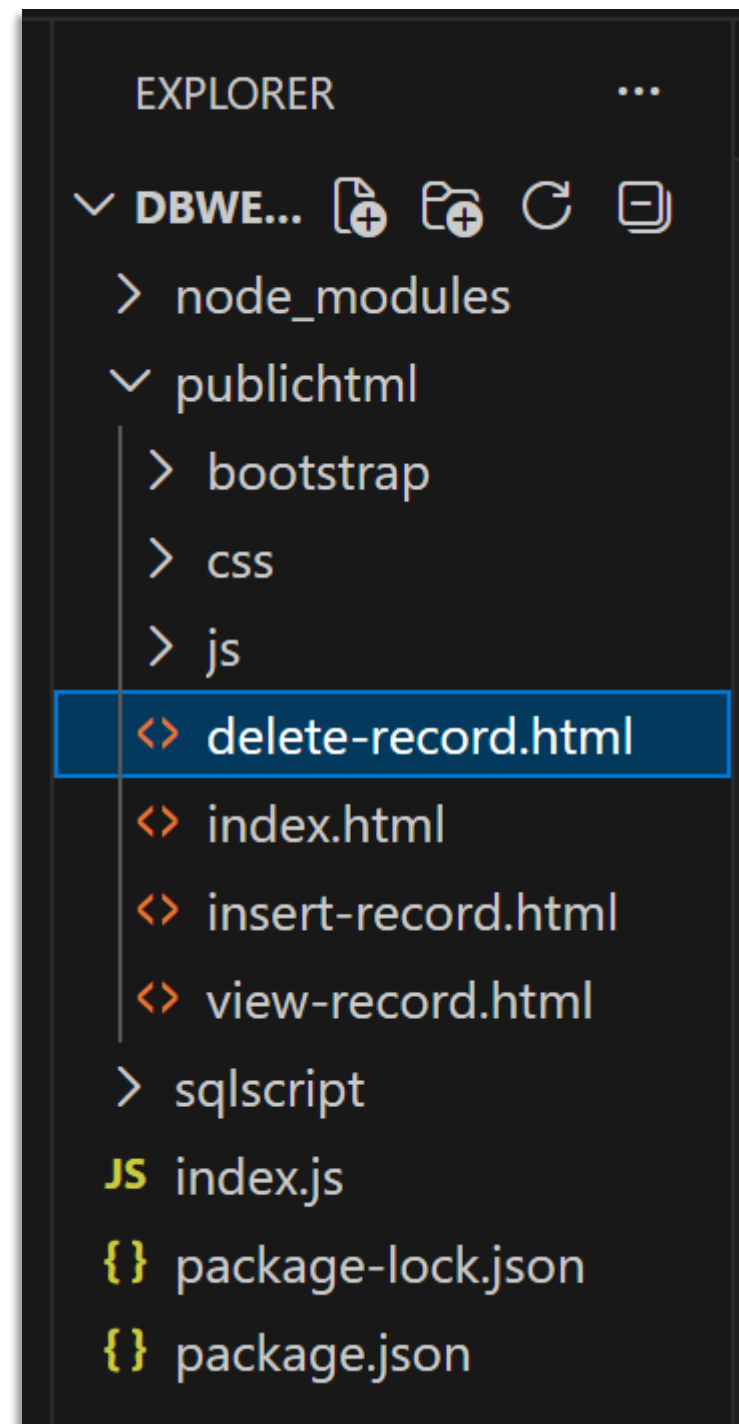
Enter Student ID:

2

Load Record

Student Management System | Full Stack Application

Delete Record



Delete Record

localhost:3000/delete-record.html

☆ f? [Color Picker] [Pencil] [Eraser] [Download] [Profile] [Menu]

☰ ★ Bookmarks Fun Courses KSA-Banks KAU Jobs Islam Journals Google Translate Suggested Sites References SDN NanoNetworks NZ Life NLP >> All Bookmarks

All RecordsView/Edit RecordDelete RecordInsert Record

Delete Student Record

Enter Student ID to Delete:

Enter student ID (e.g., 1, 2, 3...)

Load Record

Student Management System | Full Stack Application

Delete Record

[All Records](#)
[View/Edit Record](#)
[Delete Record](#)
[Insert Record](#)

Delete Student Record

Enter Student ID to Delete:

[Load Record](#)

Warning: This action cannot be undone. Please confirm before deleting.

Student Details

Student ID: 36	Full Name: Nour El-Din
Gender: Female	Semester: Semester 5
Major: SE	

[Permanently Delete Record](#)
[Cancel](#)

Student Management System | Full Stack Application

Delete Record

```
const API_BASE_URL = "http://localhost:3000/data";
let currentStudent = null;

function showAlert(message, type) {
  const alertDiv = document.createElement("div");
  alertDiv.className = `alert alert-${type} alert-dismissible fade show position-
fixed top-0 end-0 m-3`;
  alertDiv.style.zIndex = "9999";
  alertDiv.style.minWidth = "300px";
  alertDiv.innerHTML = `${message}<button type="button" class="btn-close" data-bs-
dismiss="alert"></button>`;
  document.body.appendChild(alertDiv);
  setTimeout(() => alertDiv.remove(), 3000);
}
```

Delete Record

```
document.addEventListener("DOMContentLoaded", () => {  
  const urlParams = new URLSearchParams(window.location.search);  
  const id = urlParams.get("id");  
  if (id) {  
    document.getElementById("studentId").value = id;  
    loadStudent();  
  }  
});
```

Delete Record

```
async function loadStudent() {
  const id = document.getElementById("studentId").value;
  if (!id) {
    showAlert("Please enter a student ID", "warning");
    return;
  }

  try {
    showLoading(true);
    const response = await fetch(`${API_BASE_URL}/student/${id}`);
    const result = await response.json();

    if (result.success) {
      currentStudent = result.data;
      displayStudentDetails(result.data);
      showAlert(
        "Student record loaded. Please review before deleting.",
        "warning",
      );
    } else {
```

Delete Record

```
function displayStudentDetails(student) {
  document.getElementById("displayId").innerHTML =
    `<strong>${student.id}</strong>`;
  document.getElementById("displayName").innerHTML = student.name;
  document.getElementById("displayGender").innerHTML =
    student.gender || "N/A";
  document.getElementById("displaySemester").innerHTML =
    `<span class="badge bg-secondary">Semester ${student.semester}</span>`;
  document.getElementById("displayMajor").innerHTML =
    `<span class="badge ${student.major === "CS" ? "bg-primary" : student.major === "IS" ?
"bg-success" : "bg-danger"}">${student.major}</span>`;
  document.getElementById("studentDetails").style.display = "block";
}
```


Delete Record

```
async function deleteStudent() {  
  if (!currentStudent) return;  
  
  const confirmed = confirm(  
    `Are you ABSOLUTELY sure you want to delete ${currentStudent.name} (ID:  
    ${currentStudent.id})?\n\nThis action cannot be undone!`,  
  );  
  
  if (confirmed) {  
    try {  
      showLoading(true);  
      const response = await fetch(  
        `${API_BASE_URL}/student/${currentStudent.id}`,  
        {  
          method: "DELETE",  
        },  
      );  
      const result = await response.json();  
  
      if (result.success) {
```

ID	Name	Gender	Semester	Major	Actions
1	Omar Al-Farsi	Male	Semester 3	CS	Edit Delete
2	Fatima Hassan	Female	Semester 5	IS	Edit Delete
3	Ahmed Al-Mansouri	Male	Semester 1	SE	Edit Delete
4	Layla Abdullah	Female	Semester 7	CS	Edit Delete
5	Youssef Khalil	Male	Semester 2	IS	Edit Delete
6	Nour El-Din	Female	Semester 4	SE	Edit Delete
7	Hassan Al-Rashid	Male	Semester 6	CS	Edit Delete
8	Amina Al-Zahrani	Female	Semester 8	IS	Edit Delete
9	Kareem Abbas	Male	Semester 3	SE	Edit Delete
10	Mariam Al-Saud	Female	Semester 5	CS	Edit Delete
11	Abdullah Al-Otaibi	Male	Semester 4	IS	Edit Delete
12	Sara Al-Malki	Female	Semester 2	CS	Edit Delete
13	Mohammed Al-Ghamdi	Male	Semester 7	SE	Edit Delete
14	Reem Al-Shammari	Female	Semester 1	IS	Edit Delete
15	Faisal Al-Qahtani	Male	Semester 6	CS	Edit Delete
16	Huda Al-Harbi	Female	Semester 3	SE	Edit Delete
17	Turki Al-Dossari	Male	Semester 8	IS	Edit Delete

Reference

- <https://www.npmjs.com/package/mysql>
- https://www.w3schools.com/nodejs/nodejs_mysql.asp